

แบบฝึกหัดปฏิบัติการคาบที่ 2: Class and Object

2. จงหาข้อผิดพลาดของส่วนของโปรแกรมต่อไปนี้ โดยวงกลมตำแหน่งที่ผิด เขียนหมายเลขกำกับแต่ละตำแหน่งและอธิบายเหตุผลด้านล่างว่าเหตุใดตำแหน่งดังกล่าวจึงผิด ถ้าไม่มีข้อผิดพลาดให้ตอบว่าไม่มีข้อผิดพลาด

ส่วนของโปรแกรม	ผลลัพธ์
<pre>public class ShowErrors{ public static void main(String[] args){ ShowErrors t= new ShowErrors(5); } }</pre>	class นี้ไม่มี constructor ที่จะรับตัวแปรประเภทตัวเลข
<pre>public class ShowErrors{ public static void main(String[] args){ ShowErrors t= new ShowErrors(); t.x(); } }</pre>	class นี้ไม่มี method x แต่ถ้าหากว่ามี method x นิยามอยู่ที่อื่น โปรแกรมนี้ไม่มีข้อผิดพลาด (แต่ก็ไม่ได้ยุติ เพราะว่า class ShowErrors ถูกนิยามปิดด้วย {} ครบหมดแล้ว)
<pre>public class ShowErrors{ public void method1(){ Circle c; System.out.println("What is radius "+ c.getRadius()); c=new Circle; } }</pre>	ไม่สามารถเรียก getRadius() ผ่านตัวแปร c ได้ เพราะ c ยังไม่ได้อ้างอิง object และ บรรทัดต่อมาผิด เพราะไม่มีวงเล็บหลัง Circle
<pre>public class ShowErrors{ public static void main(String[] args){ C c = new C(5.0); System.out.println(c.value); } } class C{ int value=2; }</pre>	class C ไม่มี constructor แต่สามารถ print ได้ ถ้าหากว่านำ 5.0 ออกจาก constructor ใน new

3. จงอธิบายการทำงานของโปรแกรมต่อไปนี้

โปรแกรม	แสดงผลลัพธ์จากโปรแกรม
<pre>public class Test{ public static void main(String[] args){ Count myCount = new Count(); int times=0; for (int i=0; i<100 ;i++) increment(myCount, times); System.out.println("count is "+myCount.count); System.out.println("times is "+times); } public static void increment(Count c, int times){ c.count++; times++; } } public class Count{ public int count; public Count(int c){ count =c; } public Count(){ count =1; } }</pre>	count is 101 times is 0

3.1 ข้อแตกต่างของการส่งค่าพารามิเตอร์ของ Primitive type และการส่งค่าพารามิเตอร์ของ reference type จากโปรแกรมต่อไปนี้

แบบ primitive type คือการส่ง **ค่า** ของตัวแปรนั้นไปให้ method เพื่อใช้เฉย ๆ แต่แบบ reference type คือการส่ง **ที่อยู่** ของค่า นั้น ๆ ในที่นี้ ที่อยู่คือ object ส่งผลให้ตอนเกิดการแก้ไขค่า แบบ primitive type จะไม่เกิดการแก้ไขอะไรทั้งสิ้น แต่แบบ reference type จะมีการแก้ค่าต้นฉบับใน object เกิดขึ้น

3.2 ระบุจุดที่มีการส่งค่าพารามิเตอร์ของ Primitive type และการส่งค่าพารามิเตอร์ของ reference type

<pre>Count myCount = new Count(); int times=0; for (int i=0; i<100 ;i++) increment(myCount, times);</pre>	myCount คือ reference type โดยค่าด้านใน myCount จะถูกอ้างอิงจาก myCount times คือ primitive type
--	--

4. จงแสดง output จากโปรแกรมต่อไปนี้พร้อมอธิบายการทำงานในแต่ละบรรทัด

ส่วนของโปรแกรม	ผลลัพธ์
<pre>public class Test { public static void main(String[] args) { int[] a = {1, 2}; // ประกาศอาเรย์ 1 มิติของตัวเลข swap(a[0], a[1]); // เรียกใช้ฟังก์ชัน โดยโยนค่าของ a[0], a[1] เข้าไป System.out.println("a[0] = " + a[0] + " a[1] = " + a[1]); } public static void swap(int n1, int n2) { int temp = n1; // ประกาศตัวแปรชั่วคราว ให้มีค่าเป็น n1 (ซึ่งเป็นค่าที่รับมา) n1 = n2; // ให้ n1 มีค่าเป็น n2 (ซึ่งเป็นค่าที่รับมา) n2 = temp; // ให้ n2 มีค่าเป็น temp } }</pre>	a[0] = 2 a[1] = 1
<pre>public class Test { public static void main(String[] args) { T t1 = new T(); // สร้าง object T ใหม่ เป็น t1 T t2 = new T(); // สร้าง object T ใหม่ เป็น t2 System.out.println("t1's i = " + t1.i + " and j = " + t1.j); System.out.println("t2's i = " + t2.i + " and j = " + t2.j); } } class T { static int i = 0; // เป็นตัวแปรของ class นี้โดยเฉพาะ (static), i นี้จะไม่เป็นของ object T เมื่อถูกสร้าง แต่เป็นของ class T, หรือก็คือ ทุก object T จะมีตัวแปร i นี้เท่ากันหมดทุกตัว เสมอไป int j = 0; // เป็นตัวแปรตัวเลขของ object T() { // constructor แห่ง object i++; j = 1; } }</pre>	t1's i = 2 and j = 1 t2's i = 2 and j = 1

8. (Geometry: *n*-sided regular polygon) In an *n*-sided regular polygon, all sides have the same length and all angles have the same degree (i.e., the polygon is both equilateral and equiangular). Design a class named **RegularPolygon** that contains:

- A private **int** data field named **n** that defines the number of sides in the polygon with default value **3**.
- A private **double** data field named **side** that stores the length of the side with default value **1**.
- A private **double** data field named **x** that defines the x-coordinate of the polygon's center with default value **0**.
- A private double data field named **y** that defines the y-coordinate of the polygon's center with default value **0**.
- A no-arg constructor that creates a regular polygon with default values.
- A constructor that creates a regular polygon with the specified number of sides and length of side, centered at (0, 0).
- A constructor that creates a regular polygon with the specified number of sides, length of side, and (x,y) coordinates.
- The accessor and mutator methods for all data fields.
- The method **getPerimeter()** that returns the perimeter of the polygon.
- The method **getArea()** that returns the area of the polygon. The formula computing the area of a regular

polygon is
$$\text{area} = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}$$

Draw the UML diagram for the class and then implement the class. Write a test program that creates three **RegularPolygon** objects, created using the no-arg constructor, using **RegularPolygon(6, 4)**, and using **RegularPolygon(10, 4, 5.6, 7.8)**. For each object, display its perimeter and area.

